Taller Práctico de MongoDB

Operaciones CRUD con Python y Atlas desde VS Code

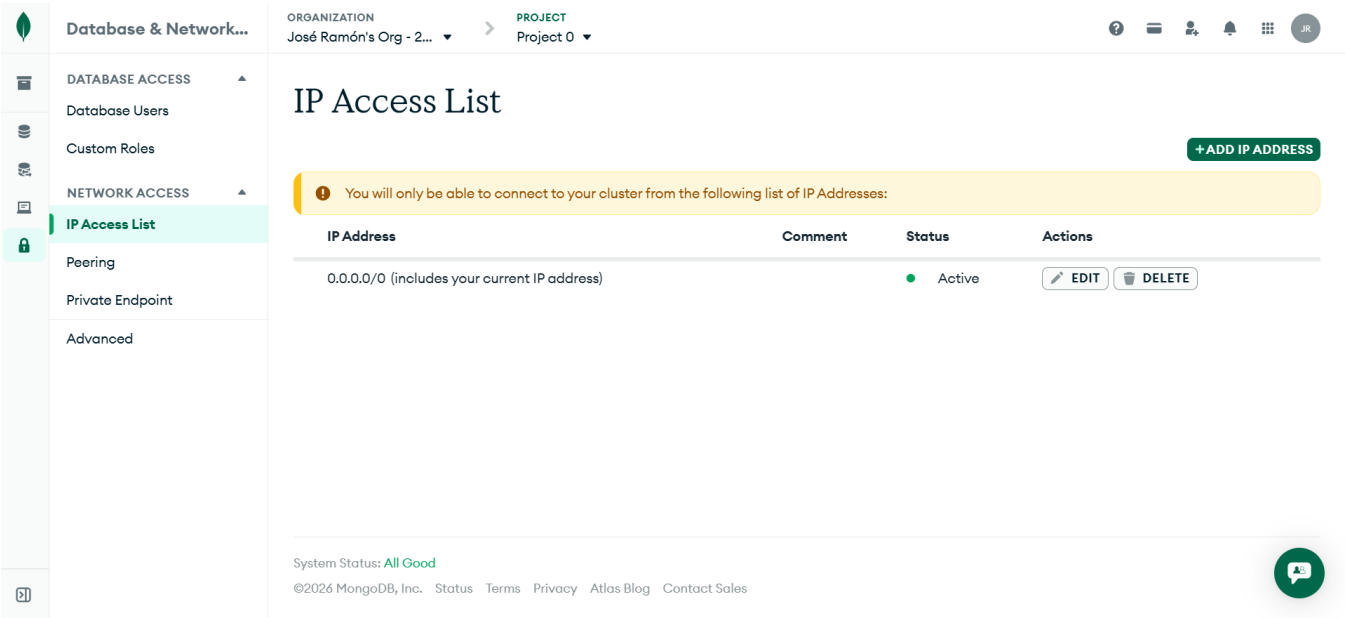
Versión CON Soluciones

0. Introducción y Configuración del Entorno

En este taller aprenderás a trabajar con MongoDB Atlas desde Visual Studio Code utilizando Python y el driver oficial pymongo. Realizarás operaciones de gestión de bases de datos, colecciones y documentos: crear, leer, actualizar y eliminar (CRUD).

0.1 Prerrequisitos

- Python 3.8+ instalado
- Visual Studio Code con la extensión Python
- Cuenta gratuita en MongoDB Atlas (cloud.mongodb.com)
- Cluster Atlas creado y usuario de base de datos configurado
- IP desbloqueada en la lista de acceso de Atlas



0.2 Instalación del driver pymongo

```python
# En la terminal de VS Code (Ctrl+`):
pip install pymongo

# Con soporte para DNS SRV (recomendado para Atlas):
pip install "pymongo[srv]"
```

0.3 Conexión a MongoDB Atlas

Obtén tu cadena de conexión desde Atlas > Connect > Drivers. Sustituye los datos con los tuyos:

```python
import pymongo
from pymongo import MongoClient

# Cadena de conexión (sustituye <usuario>, <contraseña> y <cluster>)
URI = "mongodb+srv://<usuario>:<contraseña>@<cluster>.mongodb.net/?
retryWrites=true&w=majority"

client = MongoClient(URI)

# Verificar la conexión
print("Bases de datos disponibles:", client.list_database_names())
```

Buena práctica: Variables de entorno

Es recomendable guardar la URI en una variable de entorno o en un archivo .env para no exponer
credenciales en el código. Puedes usar la librería python-dotenv:

```
from dotenv import load_dotenv; import os
load_dotenv(); URI = os.getenv("MONGO_URI")
```

MÓDULO 1 — Gestión de Bases de Datos

1. Gestión de Bases de Datos

En MongoDB las bases de datos se crean de forma implícita al insertar el primer documento. Igualmente, se eliminan automáticamente cuando se borran todas sus colecciones.

1.1 Listar las bases de datos existentes

```python
client = MongoClient(URI)

# Obtener lista de bases de datos
dbs = client.list_database_names()
print("Bases de datos:", dbs)

# Resultado esperado:
# Bases de datos: ['admin', 'local', 'sample_mflix']
```

1.2 Crear / Seleccionar una base de datos

Acceder a una base de datos que no existe la crea automáticamente cuando se inserta el primer dato.

```python
# Seleccionar (o crear) la base de datos 'tienda'
db = client["tienda"]
# También se puede usar notación de atributo:
db = client.tienda

print("BD seleccionada:", db.name)
```

1.3 Eliminar una base de datos

```python
# CUIDADO: Elimina la BD "tienda" con todos sus datos
client.drop_database("tienda")

# Verificar que se eliminó
print("BDs restantes:", client.list_database_names())
```

Ejercicios — Módulo 1

✏ Ejercicio 1: Explorar y gestionar bases de datos

Conecta a tu cluster Atlas y realiza las siguientes operaciones de gestión de bases de datos.

1. Lista todas las bases de datos disponibles en tu cluster y muéstralas por pantalla.
2. Crea una nueva base de datos llamada 'academia'.
3. Verifica que 'academia' no aparece todavía en la lista (recuerda: aún no tiene datos).
4. Inserta un documento temporal en una colección 'prueba' de 'academia' para que se materialice.
5. Vuelve a listar las bases de datos y confirma que 'academia' ya aparece.
6. Elimina la base de datos 'academia' y comprueba que desapareció.

Solución:

```python
from pymongo import MongoClient
```

```python
URI = "mongodb+srv://..."  # tu URI
client = MongoClient(URI)

# 1. Listar BDs
print(client.list_database_names())

# 2-3. Crear y verificar (no aparece aún)
db = client['academia']
print('academia' in client.list_database_names())  # False

# 4. Materializar la BD
db['prueba'].insert_one({'temp': True})

# 5. Verificar que ahora sí aparece
print('academia' in client.list_database_names())  # True

# 6. Eliminar
client.drop_database('academia')
print(client.list_database_names())
```

MÓDULO 2 — Gestión de Colecciones

2. Gestión de Colecciones

Las colecciones son equivalentes a las tablas en bases de datos relacionales. En MongoDB son esquemáticas y flexibles: cada documento puede tener campos distintos.

2.1 Listar las colecciones de una base de datos

Python — Visual Studio Code

```python
db = client["tienda"]
colecciones = db.list_collection_names()
print("Colecciones:", colecciones)
```

2.2 Crear una colección

Python — Visual Studio Code

```python
# Método 1: Implícito al insertar datos
col = db["productos"]
col.insert_one({"nombre": "Laptop", "precio": 999.99})

# Método 2: Explícito con create_collection
db.create_collection("clientes")

# Crear con validación de esquema (avanzado)
db.create_collection("pedidos", validator={
    "$jsonSchema": {
```

```
        "bsonType": "object",
        "required": ["cliente_id", "total"],
    }
})
```

2.3 Renombrar una colección

```
Python — Visual Studio Code
# Renombrar "productos" a "articulos"
db["productos"].rename("articulos")
print("Colecciones:", db.list_collection_names())
```

2.4 Eliminar una colección

```
Python — Visual Studio Code
# Eliminar la colección "articulos"
db["articulos"].drop()

# O usando el nombre de la variable
col = db.clientes
col.drop()
print("Colecciones restantes:", db.list_collection_names())
```

Ejercicios — Módulo 2

✏ Ejercicio 2: Gestión de colecciones en una biblioteca

Crea la base de datos 'biblioteca' y gestiona sus colecciones.

1. Crea la base de datos 'biblioteca' y dentro de ella las colecciones 'libros', 'autores' y 'socios'.
2. Lista todas las colecciones de 'biblioteca' para verificar que se crearon correctamente.
3. Renombra la colección 'socios' a 'usuarios'.
4. Elimina la colección 'autores'.
5. Muestra el estado final de las colecciones en 'biblioteca'.

☑ Solución:

```
db = client['biblioteca']

# 1. Crear colecciones con un documento inicial
db['libros'].insert_one({'titulo': 'El Quijote', 'año': 1605})
db['autores'].insert_one({'nombre': 'Cervantes'})
db['socios'].insert_one({'nombre': 'Ana García'})

# 2. Listar colecciones
print(db.list_collection_names())

# 3. Renombrar
db['socios'].rename('usuarios')

# 4. Eliminar colección
```

```
db['autores'].drop()

# 5. Estado final
print('Final:', db.list_collection_names())
# Resultado: ['libros', 'usuarios']
```

✏ Ejercicio 3: Colección con validación de esquema

Crea una colección 'empleados' con validación de datos obligatorios.

1. Crea la base de datos 'empresa'.
2. Crea la colección 'empleados' con un validador JSON Schema que exija los campos: 'nombre' (string), 'departamento' (string) y 'salario' (number).
3. Intenta insertar un documento válido y otro inválido (sin salario). Observa qué ocurre.
4. Lista las colecciones de 'empresa' y muestra la información del validador.

☑ Solución:

```
db = client['empresa']

# 2. Crear con validación
db.create_collection('empleados', validator={
    '$jsonSchema': {
        'bsonType': 'object',
        'required': ['nombre', 'departamento', 'salario'],
        'properties': {
            'nombre': {'bsonType': 'string'},
            'departamento': {'bsonType': 'string'},
            'salario': {'bsonType': 'number'},
        }
    }
})

# 3. Documento válido
db['empleados'].insert_one(
    {'nombre': 'Luis', 'departamento': 'IT', 'salario': 35000}
)

# Documento inválido (lanza WriteError)
try:
    db['empleados'].insert_one({'nombre': 'Sin salario'})
except Exception as e:
    print('Error esperado:', e)
```

MÓDULO 3 — Gestión de Documentos

3. Gestión de Documentos

Los documentos son los registros individuales en MongoDB, almacenados en formato BSON (Binary JSON). Cada documento tiene un campo _id único generado automáticamente.

3.1 Insertar documentos

Insertar un documento (insert_one)

```python
col = db["productos"]

# Insertar un solo documento
resultado = col.insert_one({
    "nombre": "Teclado Mecánico",
    "precio": 79.99,
    "stock": 50,
    "categorias": ["hardware", "periféricos"]
})

print("ID generado:", resultado.inserted_id)
```

Insertar múltiples documentos (insert_many)

```python
documentos = [
    {"nombre": "Ratón Inalámbrico", "precio": 29.99, "stock": 100},
    {"nombre": "Monitor 24'''", "precio": 249.99, "stock": 20},
    {"nombre": "Auriculares BT", "precio": 59.99, "stock": 75},
]

resultado = col.insert_many(documentos)
print("IDs insertados:", resultado.inserted_ids)
```

3.2 Leer / Consultar documentos

Obtener todos los documentos (find)

```python
# find() devuelve un cursor iterable
for doc in col.find():
    print(doc)

# Obtener como lista
lista = list(col.find())
print("Total:", len(lista))
```

Buscar con filtros

```python
# Filtro exacto
doc = col.find_one({"nombre": "Teclado Mecánico"})

# Operadores de comparación: $lt, $lte, $gt, $gte, $ne
baratos = list(col.find({"precio": {"$lt": 50}}))
```

```python
# Proyección: mostrar solo ciertos campos (1=incluir, 0=excluir)
col.find({}, {"nombre": 1, "precio": 1, "_id": 0})

# Ordenar y limitar resultados
import pymongo
col.find().sort("precio", pymongo.ASCENDING).limit(3)
```

3.3 Actualizar documentos

Actualizar un documento (update_one)

```python
# $set modifica un campo; $inc incrementa un valor numérico
resultado = col.update_one(
    {"nombre": "Teclado Mecánico"},      # Filtro
    {"$set": {"precio": 89.99},           # Nuevo valor
     "$inc": {"stock": -5}}               # Decrementar stock
)
print("Modificados:", resultado.modified_count)
```

Actualizar múltiples documentos (update_many)

```python
# Aplicar un descuento del 10% a todos los productos < 50€
resultado = col.update_many(
    {"precio": {"$lt": 50}},
    {"$mul": {"precio": 0.9}}  # $mul multiplica el campo
)
print("Actualizados:", resultado.modified_count)
```

Upsert (actualizar o insertar si no existe)

```python
col.update_one(
    {"nombre": "Webcam HD"},             # Filtro
    {"$set": {"precio": 45.00, "stock": 30}},
    upsert=True                           # Crea si no existe
)
```

3.4 Eliminar documentos

Eliminar un documento (delete_one)

```python
# Elimina el primer documento que coincida con el filtro
resultado = col.delete_one({
    "nombre": "Auriculares BT"
})
print("Eliminados:", resultado.deleted_count)
```

Eliminar múltiples documentos (delete_many)

```
Python — Visual Studio Code
# Eliminar todos los productos sin stock
resultado = col.delete_many({"stock": {"$lte": 0}})
print("Eliminados:", resultado.deleted_count)

# Vaciar toda la colección (¡cuidado!)
col.delete_many({})  # Filtro vacío = todos
```

Operadores de actualización más usados

$set — Establece el valor de un campo (lo crea si no existe)
$unset — Elimina un campo del documento
$inc — Incrementa (o decrementa) un campo numérico
$mul — Multiplica el valor de un campo numérico
$push — Agrega un elemento a un array
$pull — Elimina un elemento de un array
$rename — Renombra un campo

Ejercicios — Módulo 3

Ejercicio 4: Catálogo de productos de una tienda online

Trabaja con la colección 'productos' en la base de datos 'tienda_online'.

1. Crea la BD 'tienda_online' y la colección 'productos'.
2. Inserta individualmente un producto: {'nombre': 'Libro Python', 'precio': 34.95, 'stock': 200, 'categoria': 'libros'}.
3. Inserta de una vez 4 productos más de distintas categorías (electrónica, ropa, etc.).
4. Muestra todos los documentos de la colección.
5. Busca y muestra solo los productos con precio superior a 50€.
6. Muestra únicamente los campos 'nombre' y 'precio' (sin _id) de todos los productos, ordenados de más barato a más caro.

Solución:

```
db = client['tienda_online']
col = db['productos']

# 2. Insertar uno
col.insert_one({'nombre': 'Libro Python', 'precio': 34.95,
                'stock': 200, 'categoria': 'libros'})

# 3. Insertar varios
col.insert_many([
    {'nombre': 'Smartphone', 'precio': 499, 'stock': 30, 'categoria':
'electronica'},
    {'nombre': 'Camiseta', 'precio': 19.99, 'stock': 150, 'categoria': 'ropa'},
    {'nombre': 'Tablet', 'precio': 299, 'stock': 45, 'categoria':
'electronica'},
    {'nombre': 'Zapatillas', 'precio': 65, 'stock': 80, 'categoria': 'ropa'},
])

# 4. Mostrar todos
```

```
for doc in col.find(): print(doc)

# 5. Precio > 50
for doc in col.find({'precio': {'$gt': 50}}): print(doc)

# 6. Proyección y orden
for doc in col.find({}, {'nombre': 1, 'precio': 1, '_id': 0}).sort('precio',
1):
    print(doc)
```

✏ Ejercicio 5: Actualización de inventario

Usa la colección 'productos' del ejercicio anterior para realizar actualizaciones.

1. Actualiza el precio del 'Libro Python' a 29.95€.
2. Añade el campo 'descuento': 10 a todos los productos de la categoría 'electronica'.
3. Incrementa en 50 unidades el stock de todos los productos de la categoría 'libros'.
4. Usa un upsert para añadir el producto {'nombre': 'Auriculares', 'precio': 89.99, 'stock': 60} si no existe.
5. Elimina el campo 'descuento' de todos los documentos usando $unset.
6. Muestra el estado final de todos los productos.

☑ Solución:

```
# 1. Actualizar precio
col.update_one({'nombre': 'Libro Python'}, {'$set': {'precio': 29.95}})

# 2. Añadir campo 'descuento' a electronica
col.update_many({'categoria': 'electronica'}, {'$set': {'descuento': 10}})

# 3. Incrementar stock de libros
col.update_many({'categoria': 'libros'}, {'$inc': {'stock': 50}})

# 4. Upsert
col.update_one(
    {'nombre': 'Auriculares'},
    {'$set': {'precio': 89.99, 'stock': 60}},
    upsert=True
)

# 5. Eliminar campo 'descuento'
col.update_many({}, {'$unset': {'descuento': ''}})

# 6. Estado final
for doc in col.find({}, {'_id': 0}): print(doc)
```

✏ Ejercicio 6: Gestión de una agenda de contactos

Crea una colección 'contactos' en la BD 'agenda' y realiza el ciclo completo CRUD.

1. Crea la BD 'agenda' y la colección 'contactos'.
2. Inserta 5 contactos con los campos: nombre, telefono, email, grupo ('amigos', 'trabajo' o 'familia').
3. Busca y muestra todos los contactos del grupo 'trabajo'.
4. Actualiza el teléfono de uno de los contactos.
5. Añade el campo 'favorito': True a todos los contactos del grupo 'familia'.

6. Elimina todos los contactos del grupo 'trabajo'.
7. Muestra el recuento final de contactos por grupo.

Solución:

```
db = client['agenda']
col = db['contactos']

# 2. Insertar contactos
col.insert_many([
    {'nombre': 'Ana', 'telefono': '600111222', 'email': 'ana@mail.com',
'grupo': 'amigos'},
    {'nombre': 'Luis', 'telefono': '600222333', 'email': 'luis@emp.com',
'grupo': 'trabajo'},
    {'nombre': 'María', 'telefono': '600333444', 'email': 'maria@mail.com',
'grupo': 'familia'},
    {'nombre': 'Pedro', 'telefono': '600444555', 'email': 'pedro@emp.com',
'grupo': 'trabajo'},
    {'nombre': 'Elena', 'telefono': '600555666', 'email': 'elena@mail.com',
'grupo': 'familia'},
])

# 3. Filtrar por grupo
for c in col.find({'grupo': 'trabajo'}): print(c['nombre'])

# 4. Actualizar teléfono
col.update_one({'nombre': 'Ana'}, {'$set': {'telefono': '699000111'}})

# 5. Campo favorito a familia
col.update_many({'grupo': 'familia'}, {'$set': {'favorito': True}})

# 6. Eliminar trabajo
col.delete_many({'grupo': 'trabajo'})

# 7. Recuento
for grupo in ['amigos', 'familia']:
    print(grupo, ':', col.count_documents({'grupo': grupo}))
```

✏ Ejercicio 7: Sistema de puntuaciones de videojuegos

Gestiona una colección 'puntuaciones' con operaciones avanzadas de actualización.

1. Crea la BD 'videojuegos' y la colección 'puntuaciones'.
2. Inserta 6 jugadores con los campos: jugador, juego, puntos (número), nivel y victorias (número).
3. Muestra los 3 jugadores con mayor puntuación.
4. Incrementa en 100 puntos y en 1 victoria a todos los jugadores con más de 500 puntos.
5. Usa $mul para doblar el número de puntos del jugador con más victorias.
6. Elimina los jugadores con menos de 200 puntos.
7. Muestra el ranking final ordenado por puntos descendente.

Solución:

```
db = client['videojuegos']
col = db['puntuaciones']

# 2. Insertar jugadores
col.insert_many([
```

```
    {'jugador': 'AlphaWolf', 'juego': 'Quake', 'puntos': 1200, 'nivel': 10,
'victorias': 8},
    {'jugador': 'ByteQueen', 'juego': 'Quake', 'puntos': 850, 'nivel': 7,
'victorias': 5},
    {'jugador': 'PixelKing', 'juego': 'Tetris', 'puntos': 620, 'nivel': 5,
'victorias': 3},
    {'jugador': 'NullPtr', 'juego': 'Tetris', 'puntos': 150, 'nivel': 2,
'victorias': 1},
    {'jugador': 'DataNinja', 'juego': 'Pong', 'puntos': 430, 'nivel': 4,
'victorias': 2},
    {'jugador': 'SyntaxError', 'juego': 'Pong', 'puntos': 90, 'nivel': 1,
'victorias': 0},
])

# 3. Top 3
top3 = col.find({},{'_id':0}).sort('puntos', -1).limit(3)
for p in top3: print(p['jugador'], p['puntos'])

# 4. Incrementar puntos y victorias
col.update_many({'puntos': {'$gt': 500}},
    {'$inc': {'puntos': 100, 'victorias': 1}})

# 5. Doblar puntos del más victorioso
top = col.find_one(sort=[('victorias', -1)])
col.update_one({'_id': top['_id']}, {'$mul': {'puntos': 2}})

# 6. Eliminar puntos bajos
col.delete_many({'puntos': {'$lt': 200}})

# 7. Ranking final
for p in col.find({},{'_id':0}).sort('puntos',-1):
    print(f"{p['jugador']}: {p['puntos']} pts")
```

🏆 MÓDULO 4 — Proyecto Integrador

4. Proyecto Integrador — Sistema de Gestión de una Biblioteca

Aplica todos los conocimientos adquiridos para construir un sistema completo de gestión de una biblioteca digital.

✏ Ejercicio 8: Sistema completo de biblioteca digital

Implementa un sistema de biblioteca con gestión de libros, autores y préstamos.

1. Crea la BD 'biblioteca_digital' con las colecciones: 'libros', 'autores' y 'prestamos'.
2. Inserta al menos 3 autores con: nombre, nacionalidad, año_nacimiento.
3. Inserta al menos 6 libros con: titulo, autor_nombre, año, genero, disponible (bool), copias.
4. Inserta 3 préstamos con: usuario, libro_titulo, fecha_prestamo, devuelto (bool).

5. Consulta: muestra todos los libros disponibles ordenados por título.
6. Consulta: muestra todos los préstamos no devueltos.
7. Actualización: marca como 'devuelto: True' uno de los préstamos y aumenta en 1 las copias del libro correspondiente.
8. Actualización: descuenta 1 copia y marca 'disponible: False' a los libros con copias = 0.
9. Eliminación: borra todos los préstamos devueltos.
10. Muestra el recuento total de documentos en cada colección al finalizar.

Resumen de Operaciones

Resumen de Operaciones Principales

Operación	Método pymongo	Descripción
Listar BDs	`client.list_database_names()`	Lista todas las bases de datos
Crear/seleccionar BD	`client['nombre']`	Crea o accede a una BD
Eliminar BD	`client.drop_database('nombre')`	Borra una BD completa
Listar colecciones	`db.list_collection_names()`	Lista colecciones de una BD
Crear colección	`db.create_collection('nombre')`	Crea explícitamente una colección
Renombrar colección	`col.rename('nuevo_nombre')`	Renombra una colección
Eliminar colección	`col.drop()`	Borra una colección completa
Insertar uno	`col.insert_one(doc)`	Inserta un documento
Insertar varios	`col.insert_many([docs])`	Inserta múltiples documentos
Buscar todos	`col.find()`	Devuelve cursor con todos los docs
Buscar filtrado	`col.find({'campo': valor})`	Filtra por condición
Buscar uno	`col.find_one({'campo': valor})`	Devuelve el primer resultado
Actualizar uno	`col.update_one(filtro, update)`	Actualiza el primer documento
Actualizar varios	`col.update_many(filtro, update)`	Actualiza todos los coincidentes
Eliminar uno	`col.delete_one(filtro)`	Elimina el primer coincidente
Eliminar varios	`col.delete_many(filtro)`	Elimina todos los coincidentes
Contar	`col.count_documents(filtro)`	Cuenta documentos